



Chat API client

Generate typed operations from the backend OpenAPI contract.

Problem and boundary

Public contract

Working integration

@epam/ai-dial-chat-api-client



When this library is useful

Generate typed operations from the backend OpenAPI contract.

01

Call the BFF

Use typed API classes from application adapters.

02

Evolve an endpoint

Change Swagger/DTO sources, then regenerate.

03

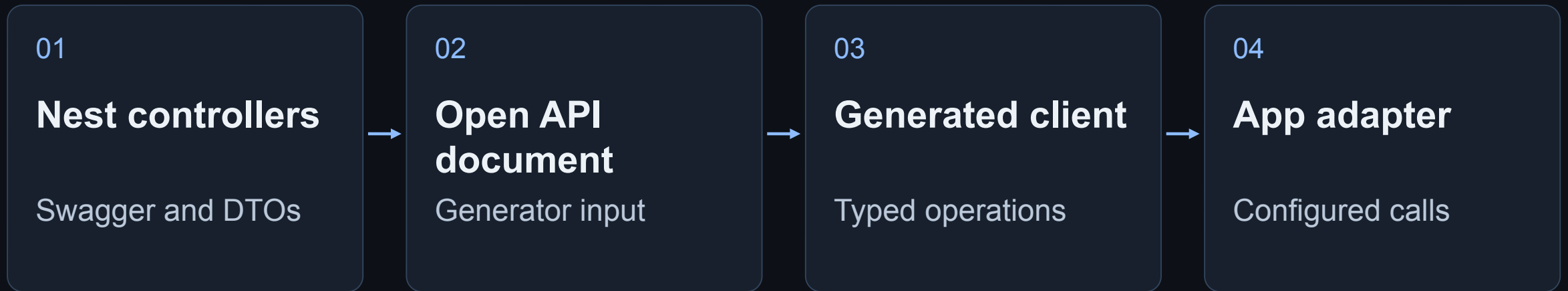
Inspect transport details

Use Raw operations when headers or the Response are needed.



Follow the ownership boundary

Inputs and effects stay explicit.



Library: Generated endpoint paths, DTOs, serializers and Fetch transport runtime.



The public contract to start with

DeploymentsApi · selected surface; full API index is in the bundle.

Input or operation	Contract
Configuration	basePath, credentials, middleware
listDeployments()	Promise<DeploymentsResponseDto>
listDeploymentsRaw({})	Raw response wrapper and value()
Generation source	Nest Swagger + openapi.json



A minimal workspace integration

TypeScript / React example · imports and host setup abridged.

```
const api = new DeploymentsApi(  
  new Configuration({  
    basePath: '',  
    credentials: 'include',  
  }),  
);  
const deployments = await api.listDeployments();
```

Full example: [examples/chat-api-client.ts](#) · uses [@epam/ai-dial-chat-api-client](#)



Wire a realistic host workflow

Keep chat api client focused on its public responsibility.

01

Configure transport

Create a stable Configuration with origin and middleware.

02

Call an operation

Use `listDeployments()` for the parsed response DTO.

03

Handle the outcome

Render the result or handle the rejected request in the host.



Preserve transport details when needed

Raw operations keep the underlying Response available.

```
const response = await api.listDeploymentsRaw({});  
const status = response.raw.status;  
const data = await response.value();
```

Focused source-backed example · see [examples/](#) and [content/api-index.md](#).



Available customization

01

Normal vs Raw

Normal methods resolve values; Raw methods expose the response wrapper.

02

Configurable transport

Configuration accepts basePath, credentials and middleware.

03

Regeneration scripts

npm run openapi and openapi:check maintain the checked-in artifacts.



Find the integration and its checks

Existing tests are evidence of intended behavior; suite results are not claimed.

Evidence	Where to look
Real consumer	usage-dashboard/src/utils/map-user-usage-to-model-limits.ts
Validation workflow	Regeneration, typecheck and downstream consumers



Constraints that affect integration

01

Never hand-edit generated code

Regeneration overwrites manual client edits.

02

Real API names

Use DeploymentsApi; the README ModelsApi example is stale.

03

Streaming needs care

Host adapters handle documented raw stream/DTO gaps.